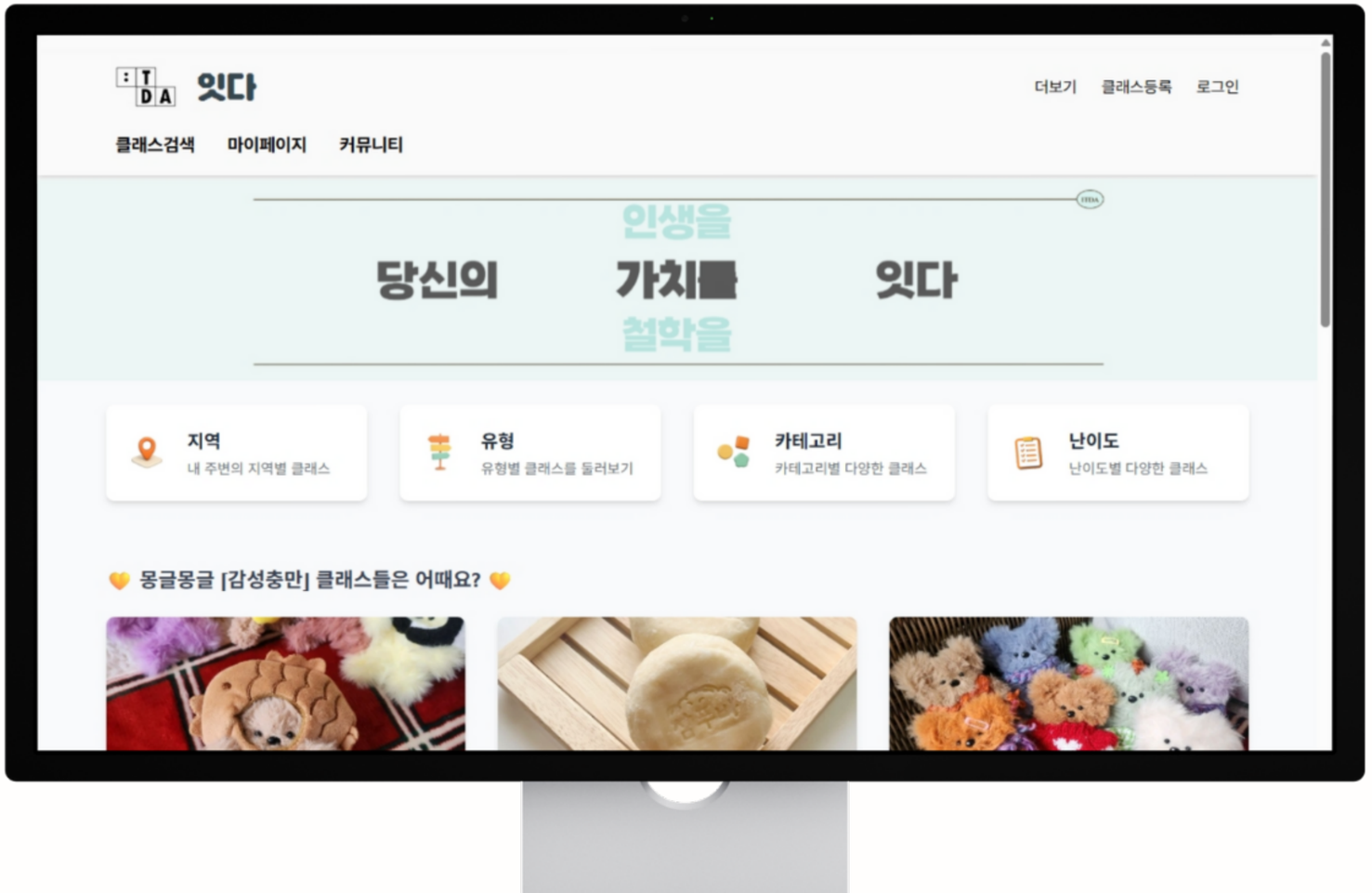
:TDA 잇다

일상에 쉼을,
재능에 가치를,
사람에 철학을 잇다

멘토-멘티 중개 플랫폼



Team. For-IntelliJ
2023136087 강보은
2020136002 최재현
2023136097 강다연

2025.06.23

프로젝트 개요



프로젝트 소개

“일상에 쉼을, 재능에 가치를, 사람에 철학을 잇다.”
재능을 나누고 사람을 연결하는 플랫폼입니다.
누구나 멘토가 되어 자신의 경험을 나누고
멘티는 관심있는 주제를 쉽게 찾아 배울 수 있습니다

잇다의 목표는
재능에 의미를 더하고, 배움이 일상 속 쉼이 되는 것입니다

개발 기간

2024년 3월 17일 ~ 6월 23일, 약 3개월 간 진행

팀 구성

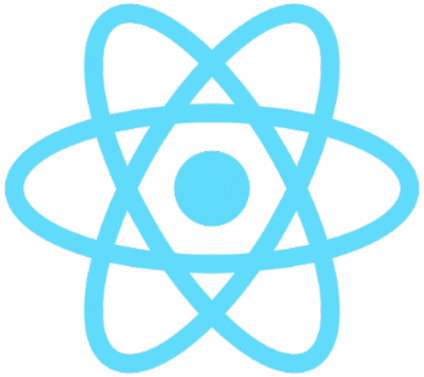
강보은: 팀장, 프론트, 백엔드
- 카카오로그인, 마이페이지 (S3 ec2연동), 메인페이지

최재현: 프론트, 백엔드
- 클래스등록, 신청, 구글api, 카카오맵스 api

강다연: 프론트, 백엔드
- 계정 권한 (로그인/로그아웃),
  프로젝트 설계(피그마,db,프론트-백 연동, MVC 설계)
  전반적인 프론트 UI (헤더,푸터, 색&폰트 등), ppt 제작

기술 스택





프론트엔드_React

- 컴포넌트 기반 구조로 재사용성과 유지보수 용이
- React Router를 사용해 페이지 전환 처리
- Axios를 통한 백엔드 API 통신 구현
- Tailwind CSS로 빠르고 일관된 UI 스타일 적용



백엔드_Spring Boot

- MVC 구조를 기반으로 컨트롤러, 서비스, 레포지토리 계층 분리
- Spring Security 없이 세션 기반 로그인 처리
- DTO 및 엔티티 분리, 유효성 검증 적용
- API 응답을 위한 RESTful 설계

기술 스택

클라우드_AWS (EC2, S3)

- EC2: Spring Boot 서버 구동
- S3: 프로필 이미지 업로드 및 저장

→ 프론트에서 업로드, URL을 서버가 받아 DB에 저장

데이터베이스_AWS RDS (MySQL)

- MySQL 기반의 안정적인 클라우드형 관계형 DB
- EC2에서 접근 가능하도록 보안 그룹 및 퍼블릭 액세스 설정
- 팀원들이 각자 계정으로 같은 DB에 접근 가능하게 구성

기술 스택

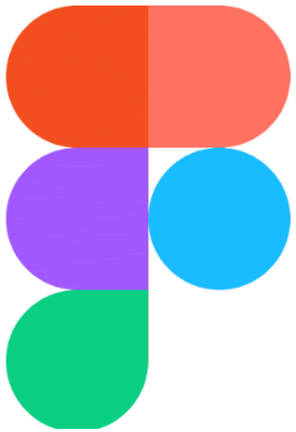
외부 API 연동_카카오,구글

Kakao 로그인 API - OAuth 2.0 방식 로그인 구현
Kakao 지도 API - 클래스 위치 확인용 지도 표시
Google Calendar API
클래스 신청 시 자동으로 일정 등록

협업관리_깃허브(버전 관리 및 협업)

- 기능 단위 브랜치 전략 사용 (feature/, fix/ 등)
- 일정 관리 및 이슈 공유를 위한 README 탭 활용

기술 스택



외부 API 연동_카카오,구글

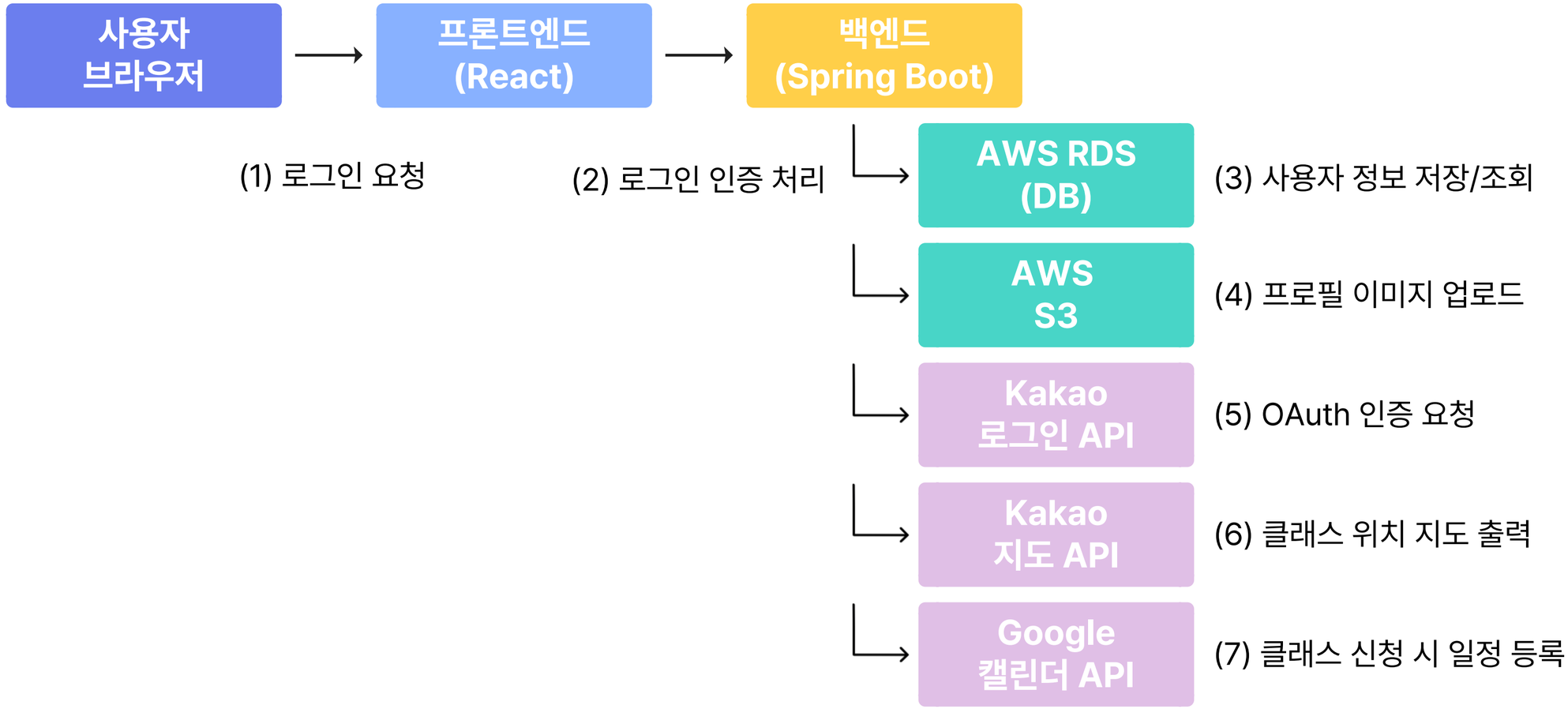
- UI 시안 설계
- 페이지 구조, 버튼 위치 등 시안 실시간 공유 및 피드백
- 디자인 통일성과 사용 흐름을 사전에 계획



협업관리_깃허브(버전 관리 및 협업)

- 팀원 간 실시간 소통을 위한 주요 채널
- 작업 진행 상황 공유, 이슈 논의, 일정 조율 등
- 음성 채널을 활용해 실시간 피드백과 회의도 진행

시스템 아키텍처


사용자
브라우저
프론트엔드
(React)
백엔드
(Spring Boot)
(1) 로그인 요청
(2) 로그인 인증 처리
AWS RDS
(DB)
(3) 사용자 정보 저장/조회
AWS
S3
(4) 프로필 이미지 업로드
Kakao
로그인 API
(5) OAuth 인증 요청
Kakao
지도 API
(6) 클래스 위치 지도 출력
Google
캘린더 API
(7) 클래스 신청 시 일정 등록

디렉토리 구조

```
frontend
├──components/ # 공통 UI 컴포넌트 (버튼, 카드 등)
├──context/ # 전역 상태 관리 (예: 로그인 상태)
├──features/ # 기능별 디렉토리
│   ├──auth/ # 로그인, 회원가입
│   ├──class/ # 클래스 등록, 신청
│   ├──community/ # 게시판, 댓글
│   ├──faq/ # 자주 묻는 질문
│   ├──main/ # 메인 페이지
│   ├──user/ # 마이페이지, 프로필
│   └──common/ # 공용 요소
│   ├──components/ # 공용 컴포넌트
│   ├──constants/ # 상수 및 경로 정의
│   └──modal/ # 공용 모달
├──services/ # API 통신 로직 (axios 기반)
└──public/
  ├──img/ # 정적 이미지
  └──fonts/ # 커스텀 폰트
```

```
src/main/java/com/itda/backend/
├──config/ # CORS, 인코딩, 예외 핸들러 설정
├──controller/ # REST API 엔드포인트
├──domain/ # JPA 엔티티 (DB 테이블 매핑)
│   └──enums/ # Enum 타입 (예: Role,
ClassLevel 등)
├──dto/ # 요청 및 응답용 DTO 클래스
├──interceptor/ # 세션 인증 처리
├──mapper/ # DTO ↔ Entity 변환 로직
├──repository/ # DB 접근 레포지토리 (JPA 기반)
└──service/ # 비즈니스 로직 처리 계층
```

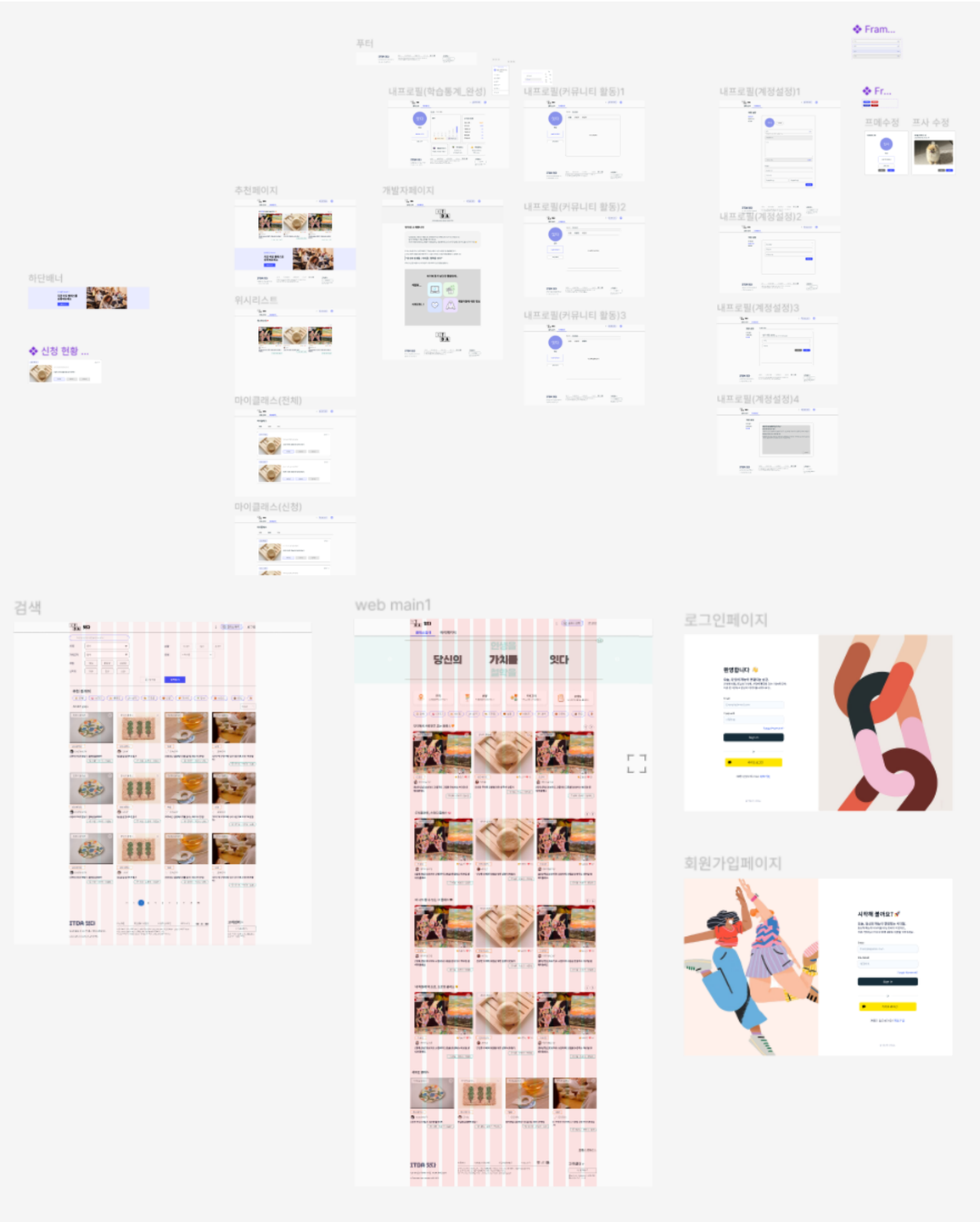
MariaDB on AWS RDS
퍼블릭 액세스 허용 + 보안 그룹 설정
EC2 및 각 팀원 로컬 PC에서 동일한 DB 접근 가능
.properties에 DB URL 및 계정 정보 설정
DDL 통일을 위해 초기 SQL 파일 공유 및 적용

UI 디자인

피그마를 이용한 협업

전체적인 톤앤매너: "따뜻하고 깔끔한 분위기"
사용 폰트와 컬러 팔레트를 미리 지정하여 일관성을 유지
주요 페이지 흐름을 실제 사용자의 사용 동선을 고려해 설계
(회원가입, 로그인, 클래스 등록, 마이페이지 등)

기획 단계에서 팀원들과 UI 시안을 공유하며 실시간 피드백



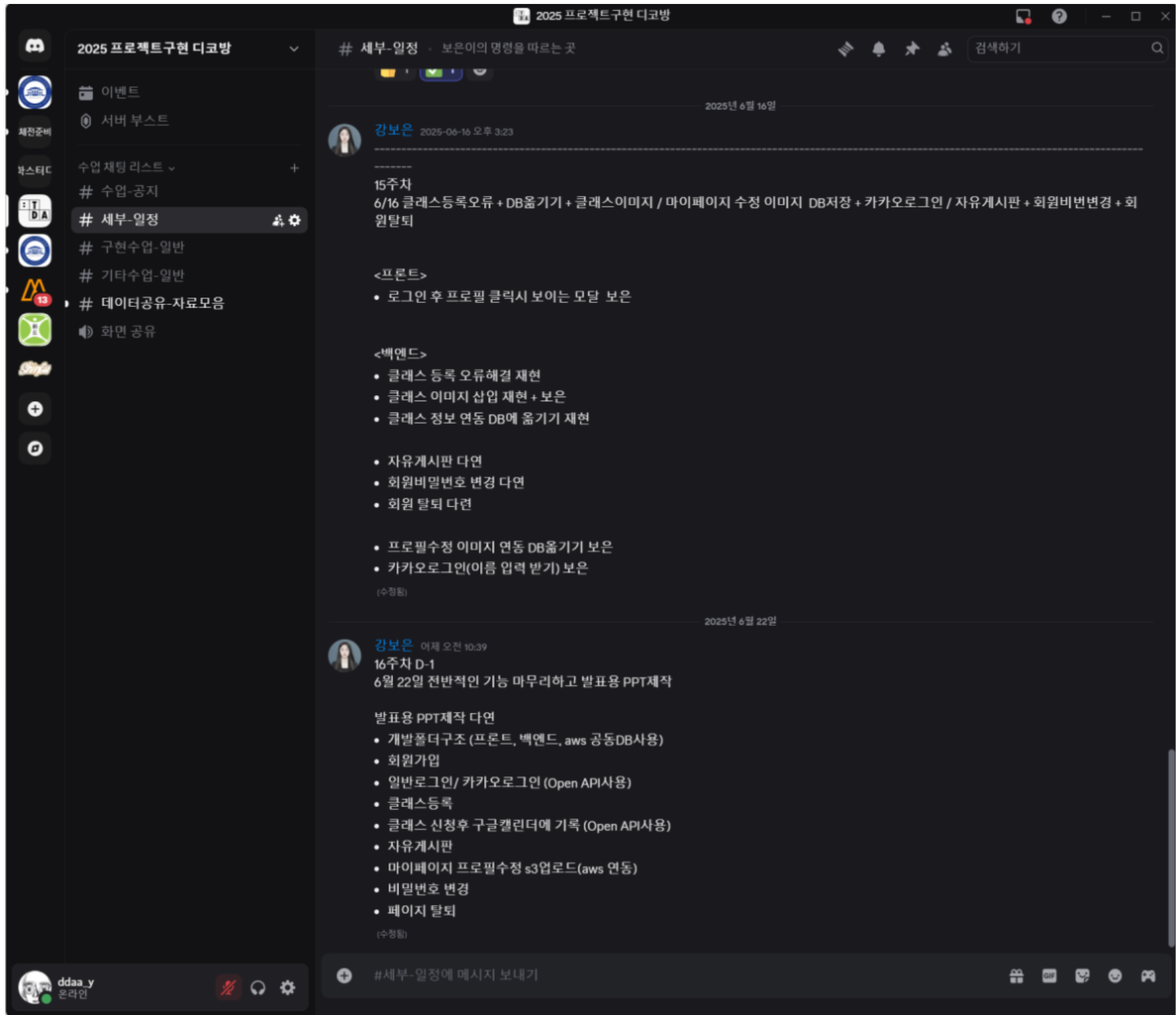
기획 및 일정관리

디스코드

전체적인 기획 논의, 일정 조율, 업무 분담을 진행
기능별로 채널을 분리해 이슈를 정리
회의 내용을 텍스트로 남겨 서로의 작업 상황을 공유

발표 준비, 클래스 기능 개발, API 연동 등
세부 작업들을 사전에 계획하고 역할을 분담하여
효율적으로 일정 관리 수행

디스코드 음성 채널을 활용해 실시간 회의 및 피드백을 진행
원활한 커뮤니케이션 환경을 구축



로그인_세션기반

구현이 간단하고 자동 로그인 유지 용이
프론트에서 토큰 관리 없이 쿠키 기반으로 유지 가능
카카오 로그인도 세션 구조로 통합되어 관리됨

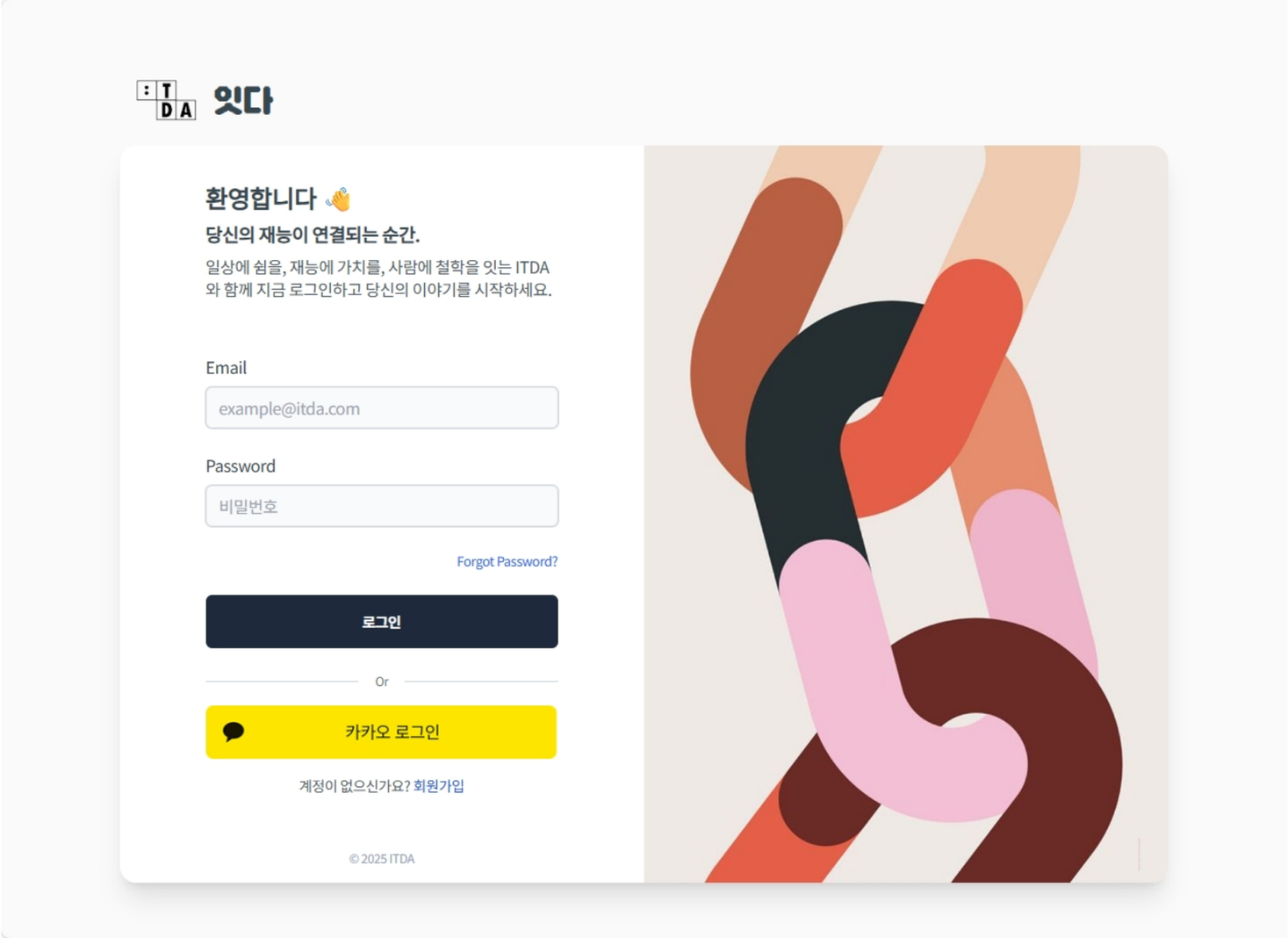
일반 로그인 흐름

입력: 이메일 + 비밀번호
서버에서 비밀번호 비교 후 세션 생성
이후 요청마다 세션을 기반으로 사용자 인증

카카오 로그인 흐름

1. 사용자가 카카오 로그인 버튼 클릭
2. 카카오 서버에서 사용자 정보 응답
3. 서버에 없으면 DB에 저장 후 자동 로그인
4. 세션 생성 → 로그인 완료

로그인 후 사용자 정보는 세션에 저장,
브라우저 쿠키에 세션 ID가 자동 포함되어
인증에 사용됨



회원가입

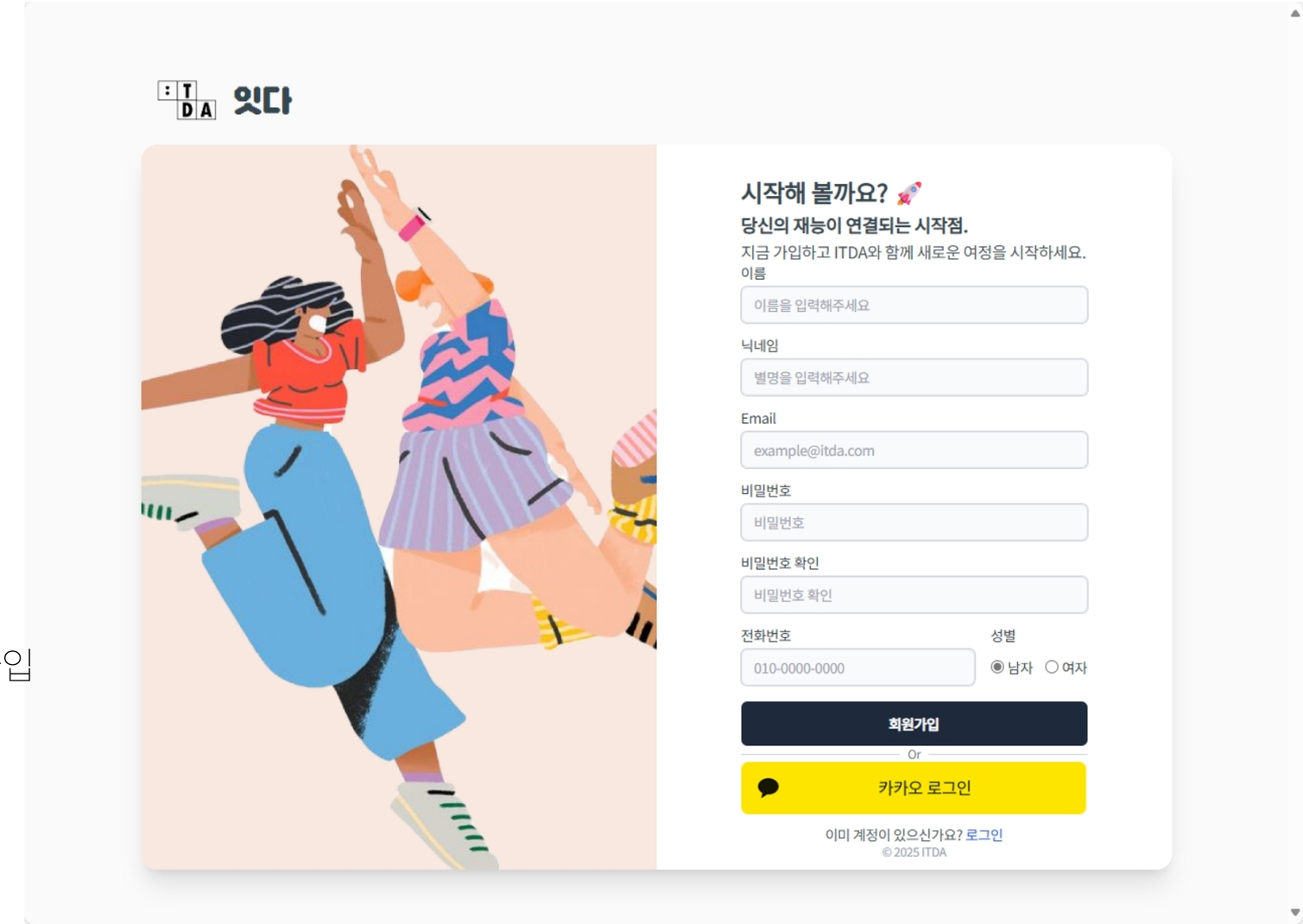
일반 회원가입

사용자는 아래 항목을 입력하여 가입
이름, 닉네임, 이메일, 비밀번호,
비밀번호 확인, 전화번호, 성별
프론트엔드에서 유효성 검사
이메일 형식, 비밀번호 일치, 빈값 여부 등
가입 후 서버에서 DB에 저장 + 세션 생성

카카오 회원가입 (OAuth 기반)

사용자는 카카오 계정으로 로그인/가입
Kakao 로그인 시:
1.카카오 서버에서 사용자 정보 응답
2. 해당 이메일이 기존 DB에 없으면 자동 가입
3. 바로 로그인 처리 및 세션 저장

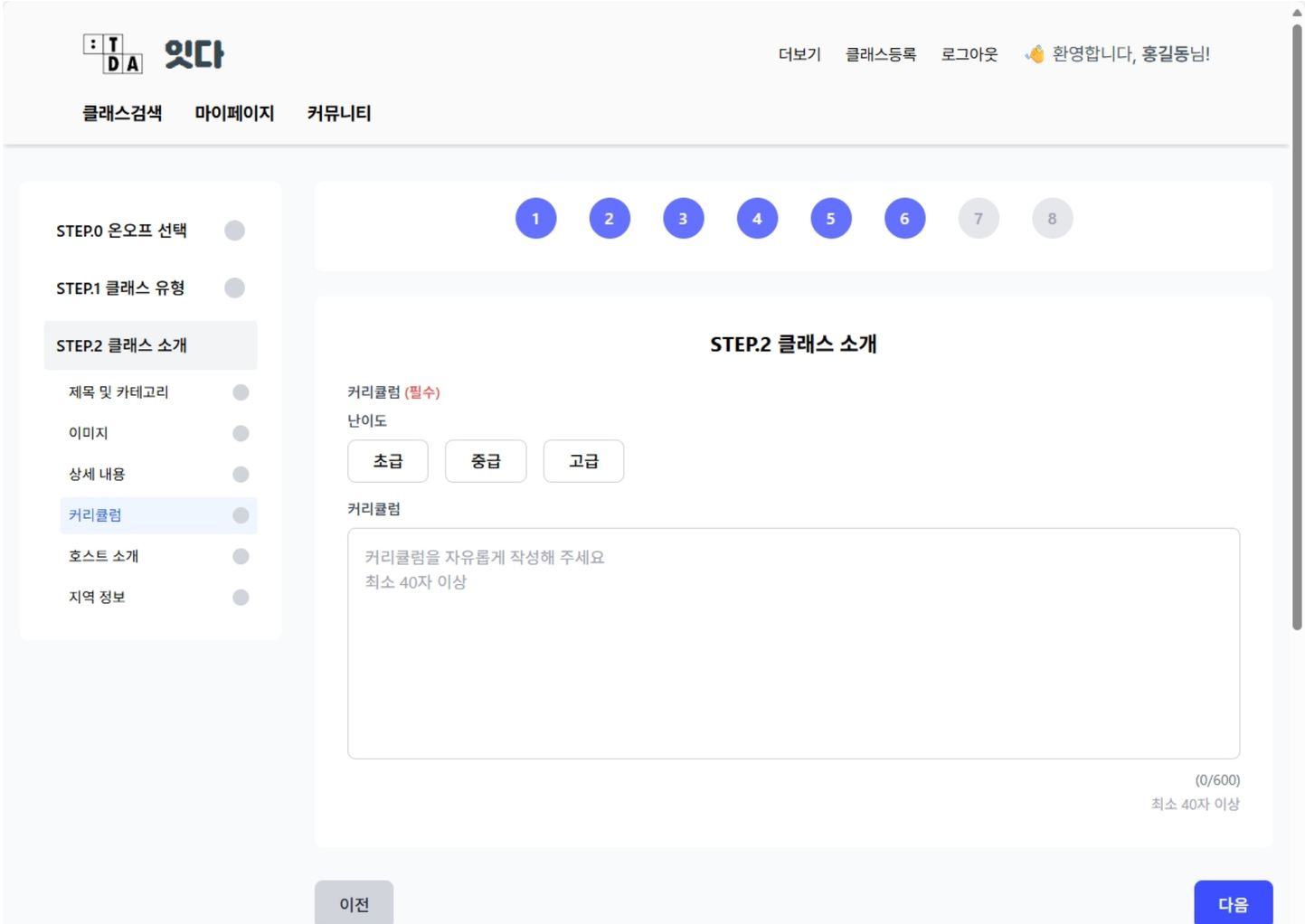
로그인과 회원가입이 동시에 처리되는 구조



클래스 등록

멘토는 로그인 후 다음 항목 입력
클래스 제목, 설명, 카테고리, 난이도,
온/오프라인 여부, 상세 커리큘럼, 대표 이미지,
위치(오프라인일 경우)

등록된 클래스는 DB에 저장,
메인 페이지 및 카테고리별로 노출됨
이미지는 S3에 업로드되어 URL로 저장됨

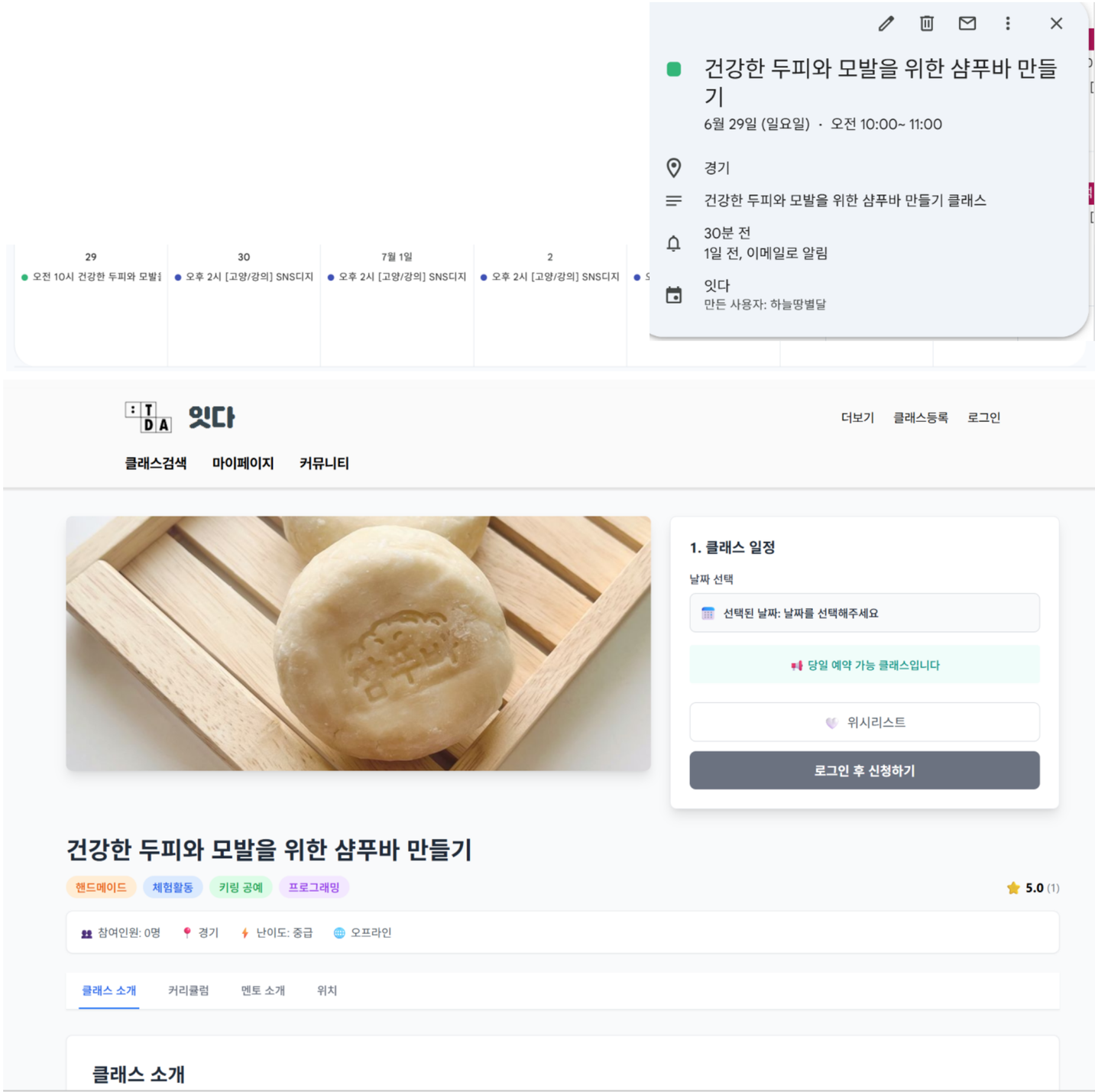


클래스 신청

멘티는 원하는 클래스를 선택 후 신청

신청 정보가 백엔드에 저장되고,
Google Calendar API를 통해 자동 등록

사용자는 캘린더에서 강의 일정을 확인 가능



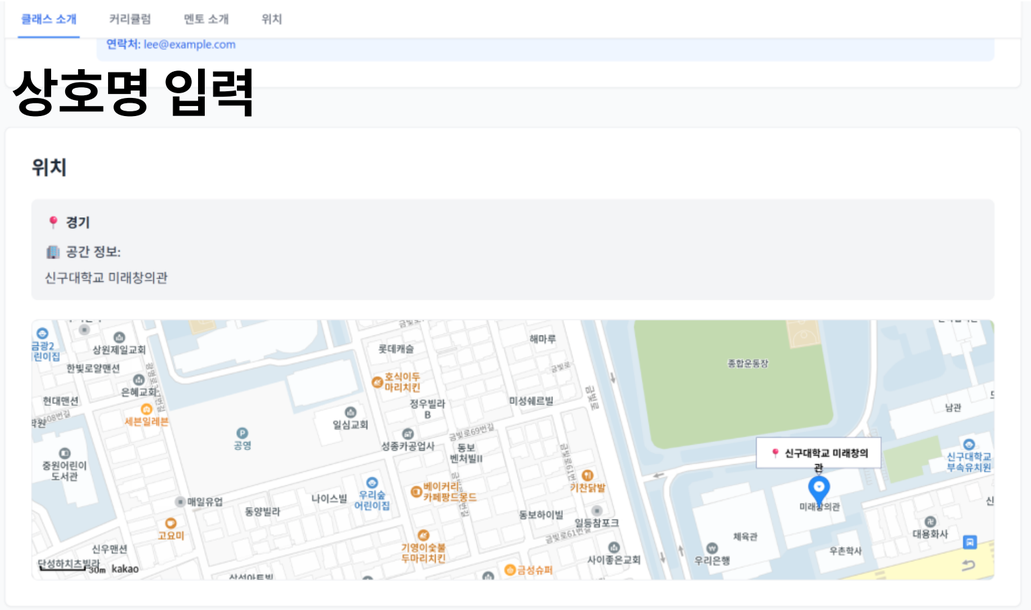
클래스 신청

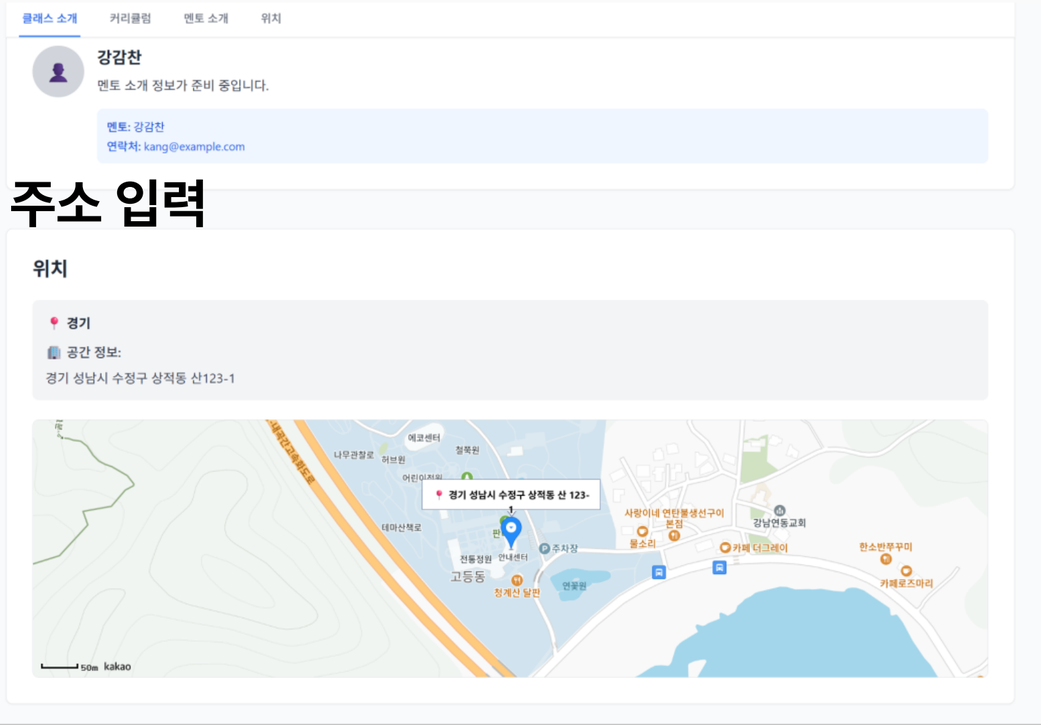
카카오맵 API

멘티는 원하는 클래스를 선택 후 신청

카카오맵스 api를 통해 위치 표시
원래는 주소값을 정확히 넣어야 지도에 위치가 뜨지만

카카오맵 api와 백엔드 코드 제어를 통해
대략적인 건물 or 상호명만 입력해도 위치가 뜨도록 함





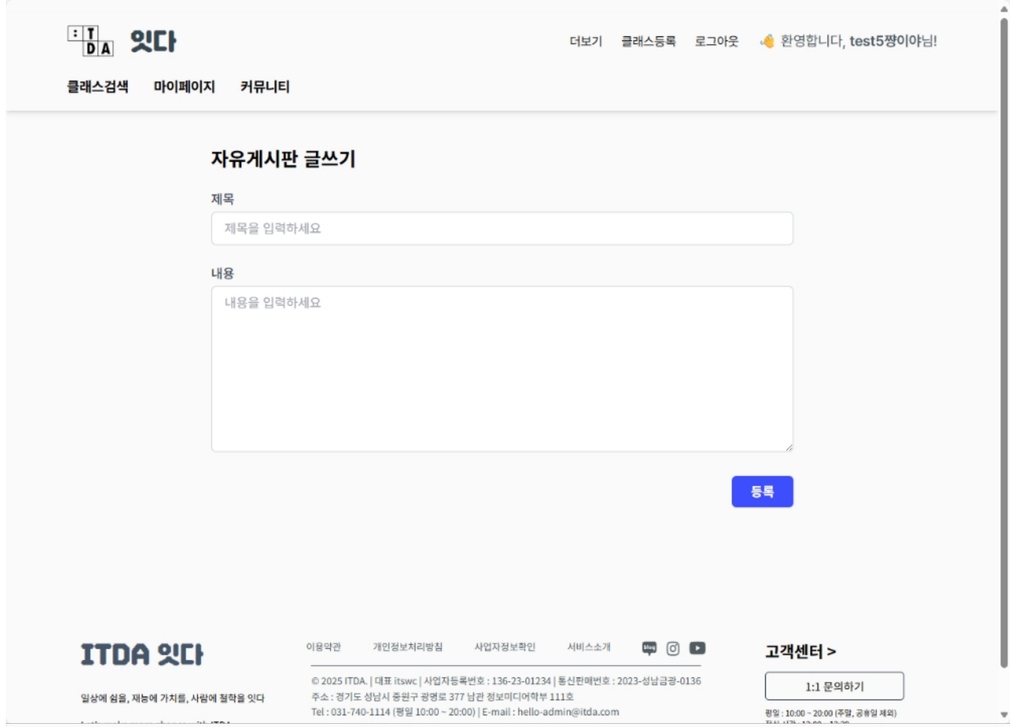
커뮤니티

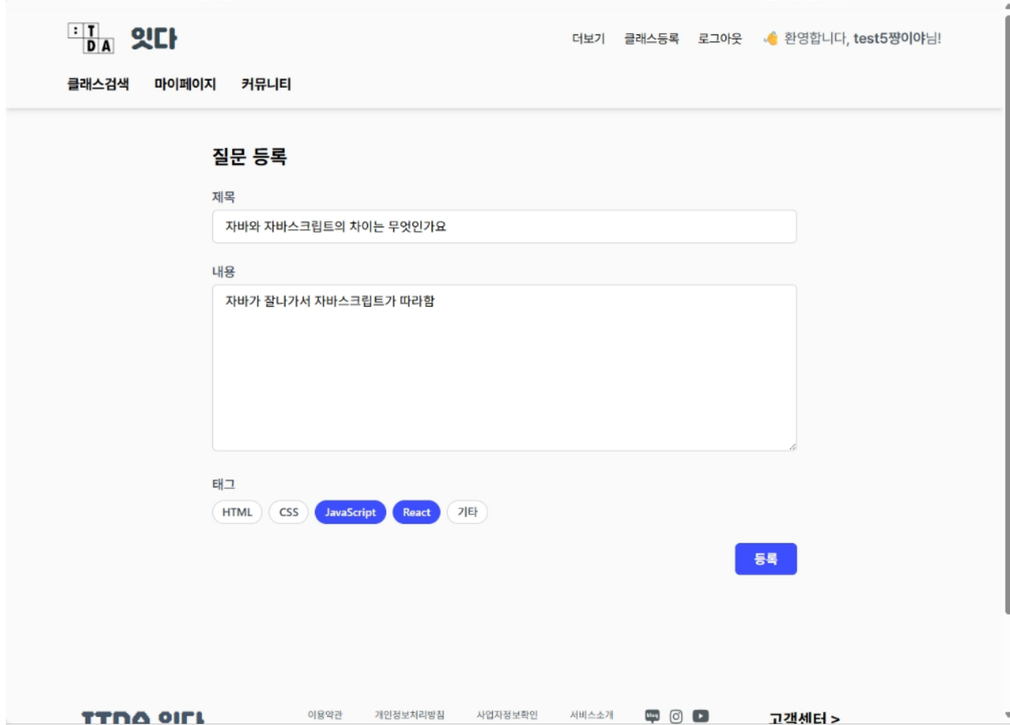
글쓰기 기능

로그인한 사용자는 자유롭게 글을 작성할 수 있음
제목, 본문 입력 가능
기본적인 유효성 검사 적용 (빈 제목 방지, 최소 글자 수 제한 등)
글 작성 시 DB에 저장되며, 최신순으로 정렬된 리스트에 노출됨
글쓴이 정보는 세션 사용자 기준으로 자동 저장

- 탭에 따라 달라지는 글쓰기 폼
같은 글쓰기 API를 사용하지만,
사용자가 어떤 탭에서 진입했는지에 따라 글쓰기 폼이 달라짐

백엔드에서는 BoardType(게시판 종류)에 따라 분기 처리
프론트에서도 같은 API (/api/board/write)를 사용하고,
게시판 종류를 함께 전송하여 목적에 따라 처리되도록 설계

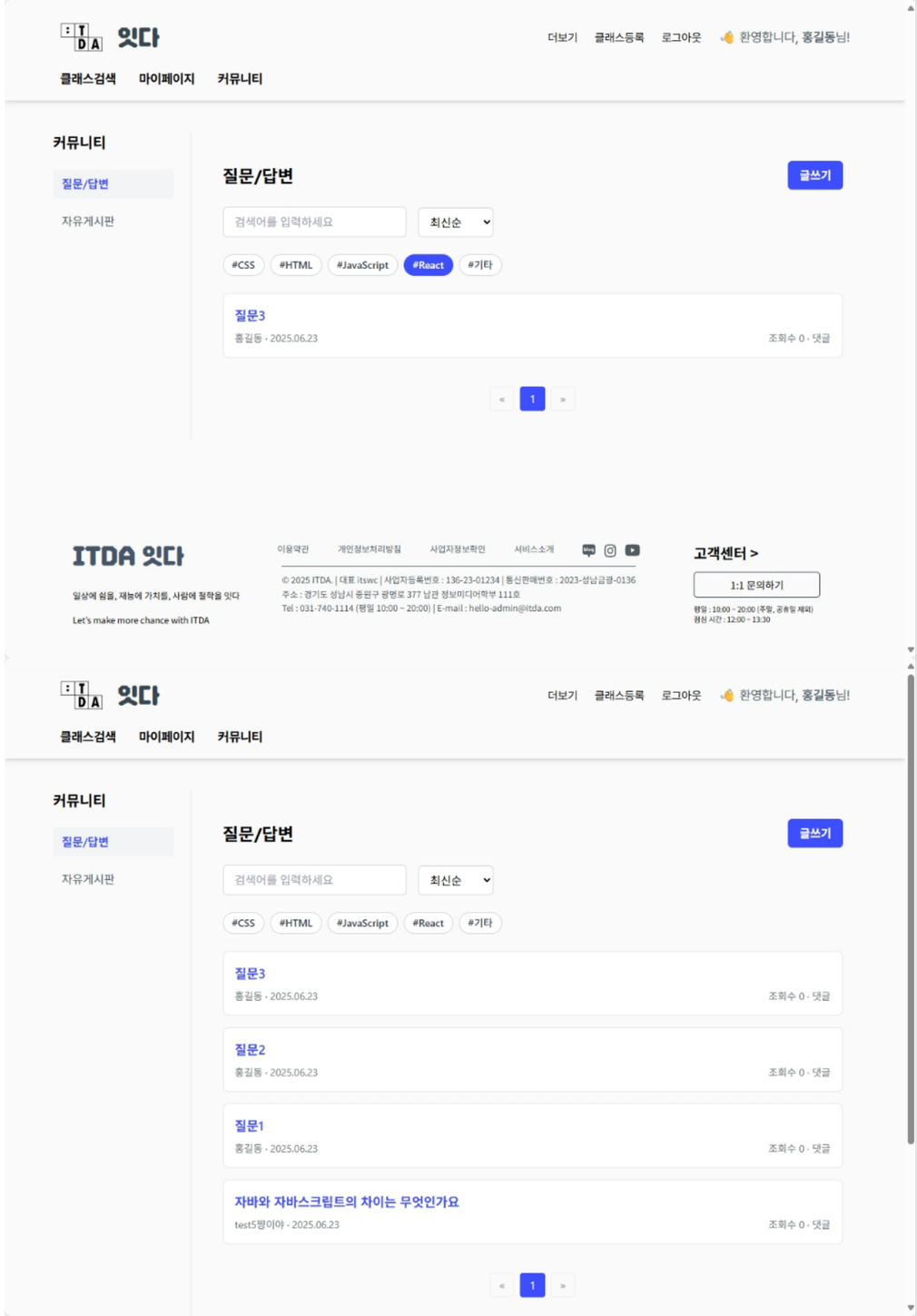




커뮤니티

DB 처리 및 리스트 출력

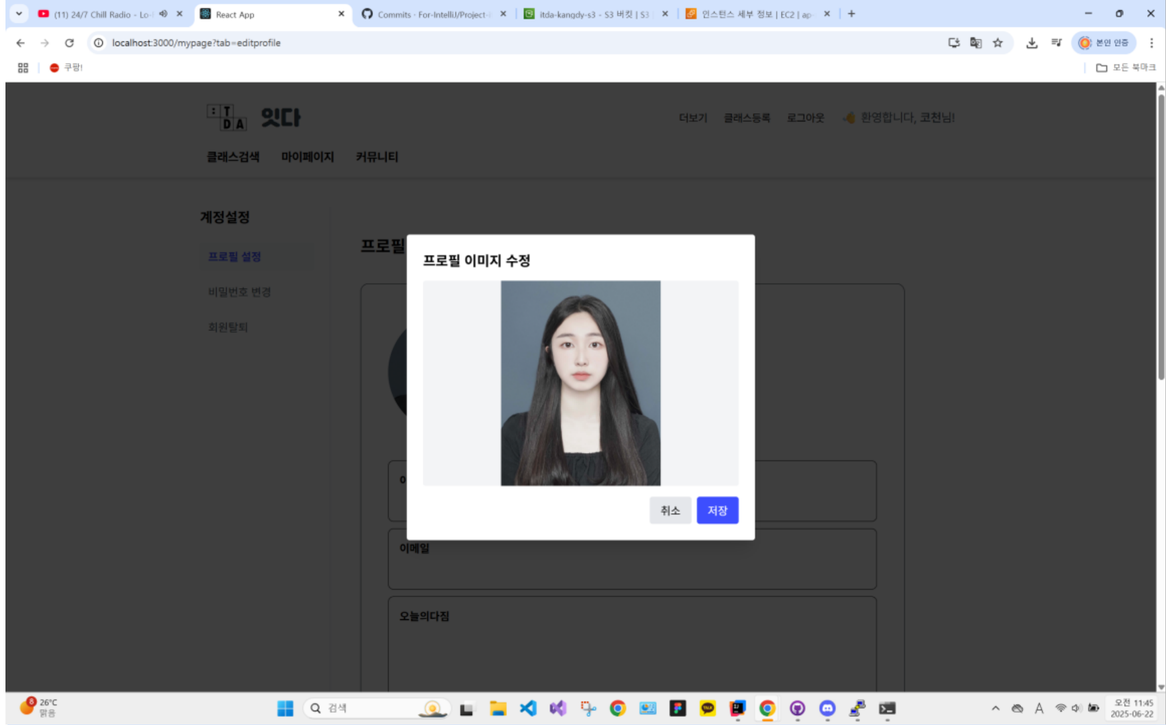
- 작성된 글은 DB에 저장되고, 최신순으로 정렬되어 게시판에 노출됨
- 게시글 카드에는 제목, 작성자 닉네임, 조회수, 댓글 수가 함께 표시됨
- 작성자 정보는 세션 사용자 기준으로 자동 저장됨
- 게시판 상단 검색바를 통해 제목·내용 검색 가능
- 태그 기능을 통해 글 분류 및 태그 기반 필터링 가능
- 태그는 글 작성 시 입력하며, 리스트와 상세 페이지에 함께 표시됨

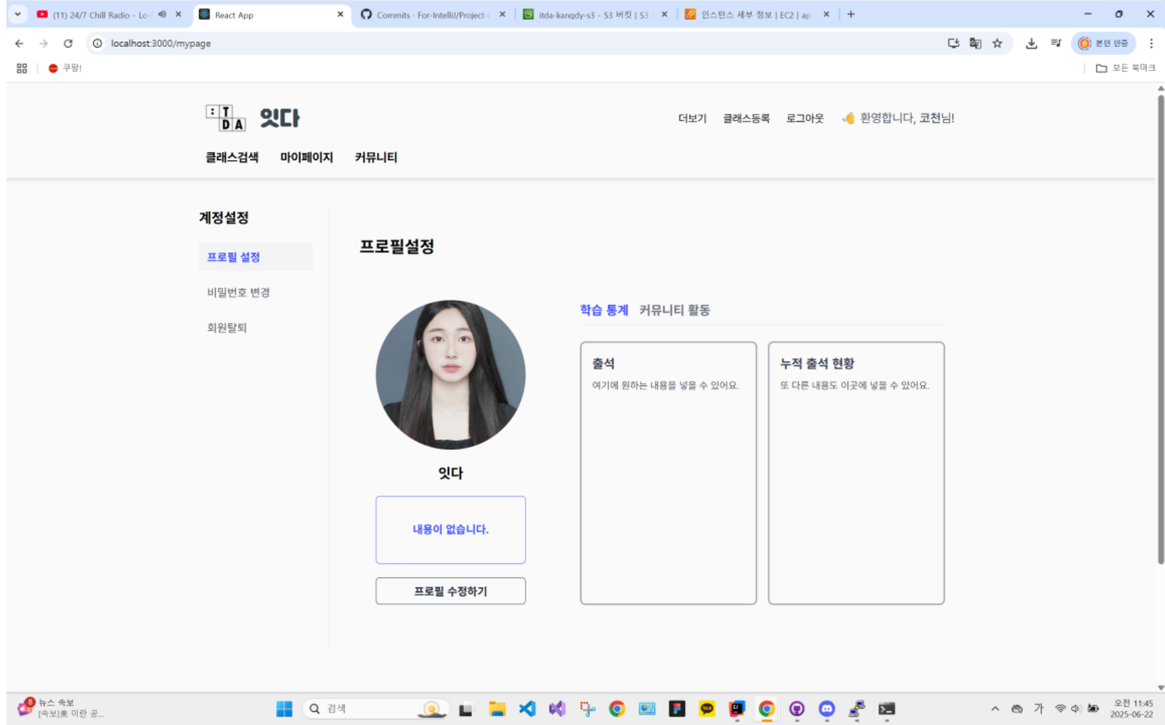


마이페이지

프로필 수정 (S3 이미지 업로드)

사용자가 마이페이지에서 닉네임, 자기소개, 프로필 수정
이미지 선택 시, 프론트에서 S3 버킷으로 직접 업로드
업로드 완료 후 S3에서 받은 이미지 URL을 서버로 전달
→ DB에 저장해당 이미지 URL로 프로필 사진 표시됨

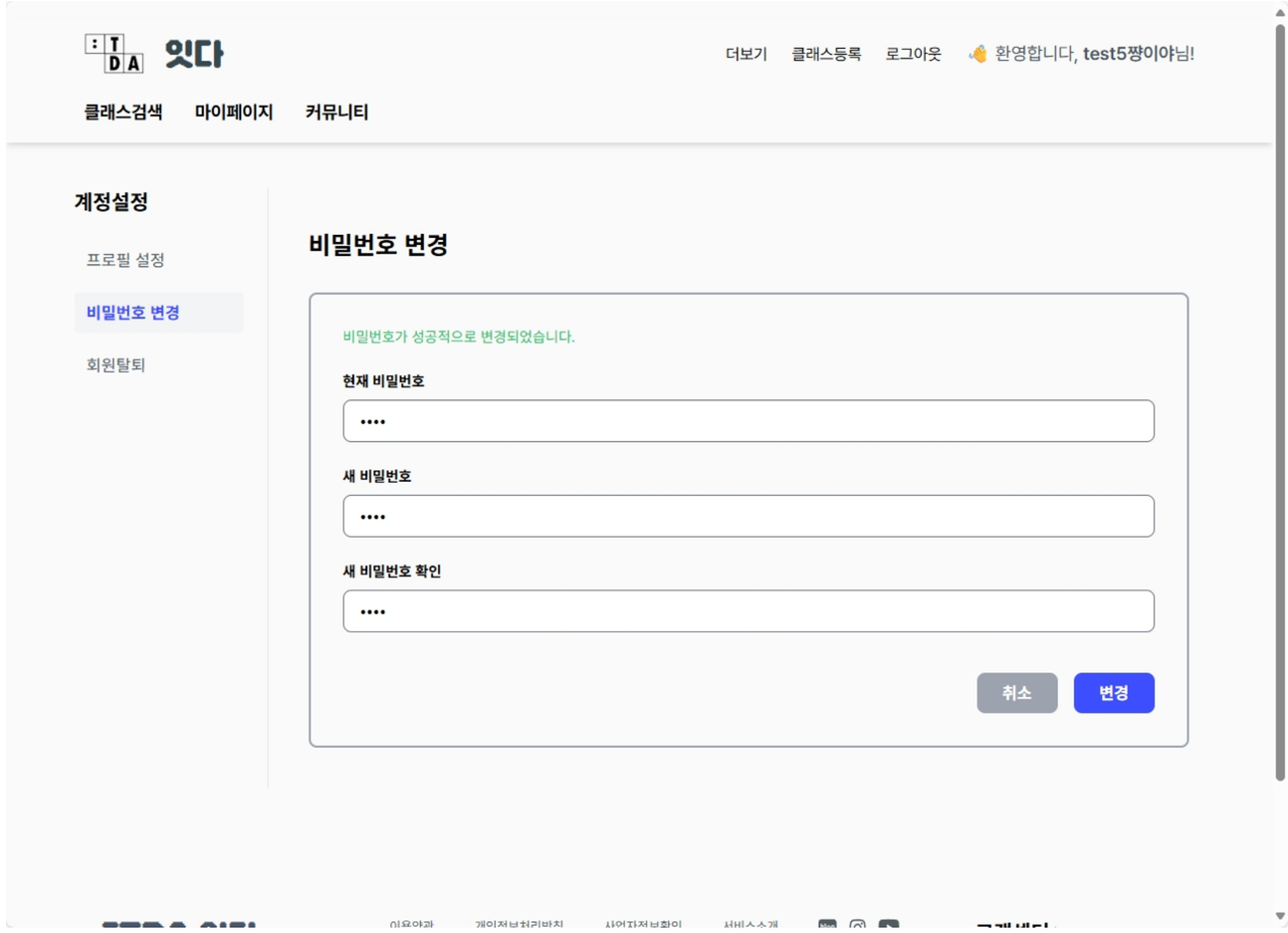




마이페이지

비밀번호 변경

사용자 본인 인증을 위해 현재 비밀번호 입력 요구
새 비밀번호를 두 번 입력 → 백엔드에서 일치 여부 검증
성공 시 "비밀번호가 변경되었습니다" 알림 표시

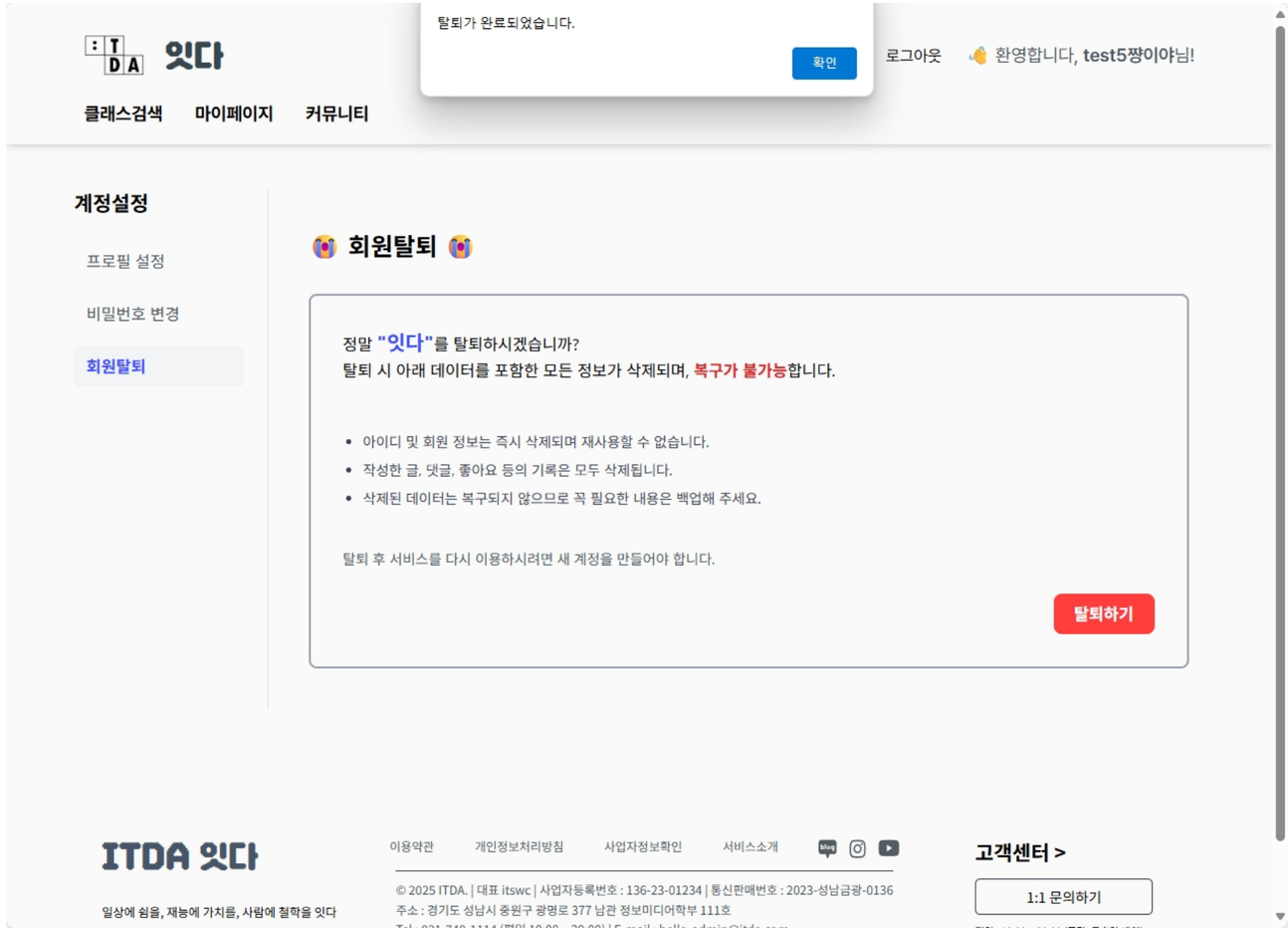


마이페이지

회원탈퇴

탈퇴 버튼 클릭 시 확인 경고창
→ 확정 시 서버에 탈퇴 요청 전송

백엔드는 해당 사용자 데이터를 삭제하고 세션 만료
→ 메인페이지로 이동



트러블슈팅 & 기술적 도전

1. EC2와 RDS 보안 그룹 설정

문제: EC2에서 RDS 접속 시 연결 오류 발생
원인: RDS 인스턴스의 보안 그룹에서 EC2의 IP를 허용하지 않음
해결: RDS 보안 그룹에 EC2 인바운드 규칙 추가(포트 3306 허용)

2. S3 업로드 시 CORS 오류

문제: 프론트에서 S3로 이미지 업로드 시 CORS 오류 발생
원인: S3 버킷의 CORS 정책 미설정
해결: S3 CORS 설정에 아래와 같은 허용 규칙 추가
(허용 Origin: *, Methods: PUT/POST/GET, Headers: *)

트러블슈팅 & 기술적 도전

3. 카카오 API 토큰 문제

문제: 카카오 로그인 중 특정 사용자에서 로그인 실패 발생
원인: 인가 코드 만료, redirect URI 불일치, 프론트와 백엔드의 인코딩 차이
해결:
redirect URI 정확히 일치시킴
인가 코드 유효 시간 고려하여 요청 타이밍 조정
인코딩 문제 해결을 위해 프론트와 백엔드 모두 UTF-8 설정 확인

4. 기타 도전

DB 구조를 직접 설계하며 외래 키와 관계 설정 오류 수정
같은 글쓰기 API를 재사용하며 게시판 종류에 따라 동적으로 처리하는 방식 구현
GitHub에서 브랜치 충돌 시 해결 경험 (merge, rebase, PR 리뷰 과정 경험)

결과&느낀점

1. 구현 기능 및 완성도

약 3개월간의 협업을 통해 회원가입, 로그인, 클래스 등록/신청, 게시판, 마이페이지 등 실사용 기능 대부분을 구현함
외부 API (카카오 로그인, 지도, 캘린더)와 AWS 인프라(S3, RDS, EC2)를 활용해 실 서비스 수준의 기능을 경험해봄
백엔드 세션 기반 로그인, S3 이미지 업로드, 게시판 정렬 및 검색, 태그 기능 등 기술적으로 깊이 있는 구현을 진행함

2. 협업을 통해 배운 점

프론트와 백엔드가 동시에 작업하기 위해 API 명세를 사전에 정리하고, 기능 단위로 역할을 나눠 효율적으로 진행함
GitHub에서 브랜치 전략(feature/...), PR 리뷰, 충돌 해결 과정을 겪으며 협업 개발 방식에 익숙해짐
공통 DB를 사용하면서 서로의 데이터 흐름과 구조를 이해하고 조율하는 경험을 통해 팀워크의 중요성을 체감함

3. 아쉬웠던 점과 개선 방향

관리자 페이지 → 추후 관리자 페이지를 통해 게시물/댓글 관리 기능 추가 예정
반응형 UI 구현 → Tailwind CSS 유틸리티를 활용한 반응형 설계 개선 예정
클래스 상세 페이지에 신청자 목록, 인원 제한 등의 기능이 추가
커뮤니티 댓글 기능 추가